EXAMEN PRÁCTICO: ESTRUCTURAS DE DATOS (TDA)

Duración: 6 horas

Ejercicio 1: Sistema de Gestión de Préstamos con Historial Reversible

Contexto: En la biblioteca principal de la universidad, el registro de préstamos de libros se realiza continuamente durante toda la jornada. Debido al alto volumen de estudiantes, el personal frecuentemente comete errores de transcripción al ingresar los datos.

Para solucionar esto, la coordinación de la biblioteca requiere un nuevo módulo de registro que permita procesar los préstamos, incorporando funciones de **"Deshacer"** (eliminar el último registro ingresado) y **"Rehacer"** (recuperar un registro eliminado accidentalmente).

Datos de entrada: El programa debe leer un archivo de texto llamado prestamos.txt que simula las acciones del bibliotecario durante el día. Cada línea del archivo comienza con un carácter que define la operación a realizar:

- P <Cédula> <CódigoLibro> <Hora>: Registra un nuevo préstamo en el sistema.
- D: Acción de "Deshacer" (anula el último préstamo válido).
- R: Acción de "Rehacer" (restaura el último préstamo anulado).

Reglas:

Utilice **apuntadores** para resolver este ejercicio.

1. Las operaciones de deshacer (D) y rehacer (R) pueden ejecutarse en cascada (varias veces consecutivas).
2. **Restricción de Historial:** Si el bibliotecario deshace una o más operaciones, y acto seguido registra un **nuevo** préstamo (P), cualquier historial de operaciones "deshechas" previamente se invalida de forma permanente y ya no podrá ser recuperado mediante la opción rehacer.
3. El sistema debe ser a prueba de fallos: si se intenta deshacer un historial vacío, o rehacer cuando no hay acciones pendientes por recuperar, el programa debe mostrar una alerta por consola y continuar leyendo el archivo.
4. Al finalizar de procesar todo el archivo, el sistema debe imprimir por pantalla el listado definitivo de préstamos que quedaron consolidados y la cantidad total de los mismos.

Ejercicio 2: Análisis de Rendimiento de Robot Logístico (Simulador de Tiempos)

Contexto: El equipo de ingeniería de un centro logístico está evaluando un nuevo modelo de robot para el almacén. En lugar de procesar una tarea completa de principio a fin, el departamento de control de calidad ha configurado al robot para trabajar bajo un sistema de "tiempos compartidos" regulado por un **Quantum**.

Esta configuración permite a los ingenieros evaluar el rendimiento del robot segmentando el esfuerzo en bloques de tiempo exactos (para este ejercicio, asuma un **Quantum** $Q = 15$ **segundos**).

Al robot se le envía una secuencia de tareas, representadas únicamente por números enteros que indican el tiempo total que requieren. El robot toma la primera tarea de la fila. Si la tarea exige 15 segundos o menos, la completa y la descarta. Si la tarea requiere más de 15 segundos, el robot invierte únicamente el tiempo del Quantum en ella, la interrumpe para reportar el rendimiento, y **vuelve a enviarla al final de la fila** exclusivamente con el tiempo que le resta por procesar.

Datos de entrada: El programa debe leer un archivo llamado tiempos.txt que contiene la lista inicial de tareas asignadas (una por línea).*(Ejemplo de contenido: 45, 10, 20)*.

Reglas

Utilice el **TDA Cola** de su autoría para resolver el ejercicio

1. El sistema debe extraer la tarea en el frente de la cola y restarle el *Quantum*. Si el tiempo restante es mayor a cero, ese nuevo valor debe ser encolado nuevamente al final de la fila. Si es menor o igual a cero, la tarea se considera finalizada y su nodo debe ser liberado de la memoria.
2. El simulador debe imprimir por consola una bitácora del procesamiento hasta que la cola de tareas quede vacía.

Salida Esperada por Consola (Basada en el ejemplo):

--- INICIANDO SIMULACIÓN DE RENDIMIENTO ---

Quantum de procesamiento: 15s

Procesando tarea de 45s. Restan 30s -> Regresa a la cola.

Procesando tarea de 10s. Terminada.

Procesando tarea de 20s. Restan 5s -> Regresa a la cola.

[...]

--- TODAS LAS TAREAS FUERON COMPLETADAS ---

Ejercicio 3: Descifrado de Comunicaciones

Contexto: El departamento de seguridad informática de la universidad ha interceptado un mensaje de red cifrado. Tras un análisis, los ingenieros descubrieron que el algoritmo utilizado es rudimentario: el orden de las palabras y los espacios se mantiene intacto, pero **las letras de cada palabra están invertidas**.

Para automatizar el descifrado, se requiere un programa que procese el mensaje carácter por carácter, apoyándose en las propiedades de las estructuras de datos para reordenar las letras.

Datos de entrada: El programa debe leer el archivo interceptado.txt que contiene el mensaje cifrado en una sola línea continua. *(Ejemplo: SOTAD ED SARUTCURTSE)*.

Reglas:

Utilice los **TDA Lista Enlazada** y **TDA Pila**. Ambas estructuras almacenarán un dato simple de tipo Carácter (char).

1. **Fase de Carga:** El programa debe leer el archivo carácter por carácter (incluyendo los espacios) y guardar cada uno al final de la Lista Enlazada. Queda terminantemente prohibido usar arreglos (como char mensaje[100]).
2. **Fase de Descifrado:** Se debe recorrer la Lista Enlazada desde el primer nodo hasta el último:
    - Si el carácter extraído **es una letra**, se debe Apilar en la Pila auxiliar.
    - Si el carácter extraído **es un espacio en blanco** (o si se llega al final de la Lista), se debe iterar Desapilando todos los caracteres de la Pila e imprimiéndolos en consola para revelar la palabra correcta.
3. No utilice funciones de librerías de cadenas (como strrev). La inversión debe ocurrir puramente por la naturaleza mecánica LIFO de la Pila.

Salida Esperada por Consola:

Extrayendo letras a la Pila auxiliar...

Espacio detectado. Palabra descifrada: DATOS

Extrayendo letras a la Pila auxiliar...

Espacio detectado. Palabra descifrada: DE

Extrayendo letras a la Pila auxiliar...

Fin de lista detectado. Palabra descifrada: ESTRUCTURAS

Mensaje original: DATOS DE ESTRUCTURAS

Ejercicio 4: Gestión Dinámica de Inventario Post-Batalla

(Evaluación: TDA Lista Enlazada Dinámica con Structs)

Contexto: En el desarrollo de un nuevo videojuego (RPG), el inventario del personaje principal está programado utilizando una **Lista Enlazada Simple**. Cada objeto (nodo) en esta lista no almacena un único valor, sino un registro detallado con las características del ítem.

Durante el combate, algunos objetos se desgastan y otros se consumen. El equipo de desarrollo necesita que usted programe la rutina de "Limpieza Post-Batalla", la cual debe recorrer el inventario, eliminar definitivamente los objetos cuya durabilidad haya llegado a cero, y generar un reporte del equipamiento.

Estructura del Nodo: El TDA Lista debe utilizar obligatoriamente el siguiente struct para los nodos de datos:

C

```
struct Item {
   char nombre[30];
   char categoria;      // 'A': Arma, 'P': Poción, 'M': Material
   int durabilidad;     // Rango de 0 a 100
   struct Item* siguiente;
};
```

Datos de entrada: El programa debe leer el archivo inventario.txt que contiene el estado del inventario tras la batalla. *(Ejemplo: Espada_Larga A 45 / Pocion_Menor P 0 / Hierro_Bruto M 100)*.

Reglas de Negocio y Requerimientos:

1. **Carga Inicial:** Leer el archivo e insertar cada objeto al final de la Lista Enlazada, rellenando todos los campos del struct.
2. **Limpieza (Manipulación de Punteros):** Recorra la lista desde el principio evaluando el campo durabilidad.
    - Si la durabilidad es igual a 0, el objeto se considera destruido. Usted debe **puentear los apuntadores** para desenganchar ese nodo de la lista y liberar esa memoria, evitando fugas (*memory leaks*).
3. **Restricción:** No se permite crear una segunda lista para copiar los objetos sobrevivientes. La eliminación debe hacerse reconectando los apuntadores de la lista original.
4. **Reporte:** Al finalizar, el programa debe leer el campo categoria de los objetos sobrevivientes e imprimir el inventario resultante, junto a un conteo de cuántas Armas, Pociones y Materiales quedaron disponibles.